
When Does Feedback Help? A Controlled Study of LLM-Guided Neural Architecture Design

Anonymous Authors
Anonymous Institution

Abstract

Large language models are increasingly used to propose neural architectures, yet it remains unclear whether iterative feedback actually improves the designs they produce. We present a controlled experiment comparing six architecture-generation strategies on CIFAR-10 and CIFAR-100: random search, validity-filtered random search, LLM zero-shot generation, LLM generation with unstructured feedback, LLM generation with structured feedback, and regularised evolutionary search. Using Qwen3-1.7B under fixed compute budgets (20 architectures per condition, 50 training epochs each, NVIDIA A100-40 GB), we find that zero-shot LLM generation significantly outperforms random search in mean accuracy (89.0 ± 0.7 vs. 77.3 ± 22.6 on CIFAR-10; Mann-Whitney $p < 0.001$, Cohen’s $d = -0.71$) while exhibiting dramatically lower variance. Remarkably, all 20 zero-shot designs share *identical* design choices (standard 3×3 convolutions, ReLU, BatchNorm, parameter std = 0 K), revealing that the LLM encodes a strong, narrow prior over a single architecture template. Contrary to prevailing assumptions, feedback does not improve upon this prior: structured feedback significantly decreases mean accuracy on CIFAR-10 ($d = +1.25$, $p_t < 0.001$, $p_U = 0.002$, Bonferroni-corrected $\alpha = 0.0071$) while increasing architectural diversity (Jaccard distance from 0.022 to 0.268). On CIFAR-100, all LLM conditions perform comparably ($p = 0.61$). These results challenge the self-refinement paradigm in LLM-based architecture search and suggest that, under fixed budgets, parallel zero-shot sampling dominates sequential refinement. Code, prompts, and all seeds will be released upon acceptance.

1 Introduction

Neural architecture search (NAS) has matured from a compute-intensive reinforcement-learning problem [Zoph and Le, 2017] into a diverse family of methods encompassing evolutionary strategies [Real et al., 2019], gradient-based relaxations [Liu et al., 2019], and one-shot approaches [Pham et al., 2018]. A recent and rapidly growing line of work replaces or augments the search algorithm with a large language model (LLM) that directly proposes architectures in response to natural-language prompts [Chen et al., 2023, Zheng et al., 2023, Nasir et al., 2024]. The appeal is intuitive: LLMs trained on vast code corpora may have internalised design patterns from thousands of published architectures, enabling them to generate competitive designs without explicit search.

A common extension of this idea is *iterative refinement*, in which the LLM receives feedback on the performance of its previous proposals and is asked to improve upon them [Madaan et al., 2023, Shinn et al., 2023]. The implicit assumption is that such feedback helps the model correct mistakes and converge toward better architectures over successive rounds. Yet evidence for this assumption is surprisingly thin. Recent theoretical and empirical analyses [Huang et al., 2024, Stechly et al., 2024, Valmeekam et al., 2023] have shown that LLM self-correction can degrade performance when the model lacks the capacity for reliable causal reasoning about its outputs, and the conditions under which feedback genuinely aids LLM-based design remain poorly characterised.

Two fundamental questions motivate this work. First, does LLM-guided generation actually outperform random search? Li and Talwalkar [2020] demonstrated that random search is a

surprisingly competitive NAS baseline, yet most LLM-NAS papers compare against sophisticated methods rather than this elementary control [Yang et al., 2020]. Without a random baseline, it is impossible to determine whether observed gains stem from the LLM’s knowledge or from properties of the search space itself. Second, does iterative feedback help, hurt, or make no difference? If LLMs cannot reliably perform causal attribution over architecture–performance relationships, feeding training metrics back into the loop may introduce systematic biases rather than useful signal.

We address both questions through a carefully controlled experiment. We define a modular CNN search space and compare six conditions that span the spectrum from fully random generation to LLM generation with two granularities of feedback, alongside a regularised evolutionary baseline. All conditions share the same search space, training protocol, and compute budget, enabling clean causal comparisons. Every condition produces zero invalid architectures, a consequence of the well-defined search space and parser sanitisation that eliminates validity as a confound. Our study yields the following findings:

- **Strong zero-shot prior.** Even without feedback, LLM zero-shot generation significantly outperforms random search on both datasets while producing architectures with *zero* structural variance: all 20 designs are identical (parameter std = 0 K, Jaccard distance 0.022).
- **Feedback hurts when the prior is strong.** On CIFAR-10, where the LLM prior is well-calibrated, both feedback conditions decrease mean accuracy ($d = 0.93$ unstructured, $d = 1.25$ structured). On CIFAR-100, all LLM conditions perform comparably ($d = -0.16$, $p = 0.61$).

- **Diversity–accuracy inversion.** Feedback increases architectural diversity (Jaccard distance from 0.022 to 0.268) but moves designs away from the LLM’s well-calibrated default, reducing accuracy.
- **Practical recommendation.** Under fixed compute budgets, parallel zero-shot sampling dominates sequential refinement on both benchmarks.

2 Related Work

Neural Architecture Search. NAS was catalysed by the reinforcement-learning approach of Zoph and Le [2017], which demonstrated that a controller network could discover competitive architectures at the cost of thousands of GPU-days. Subsequent work reduced this cost through weight sharing [Pham et al., 2018], differentiable relaxations [Liu et al., 2019], and performance predictors [White et al., 2023]. Evolutionary methods, particularly regularised evolution [Real et al., 2019], remain strong baselines that are simple to implement and often competitive [Stanley et al., 2019, So et al., 2019]. Tabular benchmarks such as NAS-Bench-201 [Dong and Yang, 2020] and NAS-Bench-101 [Ying et al., 2019] have enabled reproducible comparisons. Critically, Li and Talwalkar [2020] showed that random search with early stopping is a surprisingly strong baseline, and Yang et al. [2020] highlighted persistent methodological gaps including insufficient baselines, both of which motivate our experimental design.

LLM-Based Architecture Search. Several concurrent works have proposed using LLMs to generate or guide neural architectures. EvoPrompting [Chen et al., 2023] combines evolutionary search with LLM-based mutation operators. GENIUS [Zheng et al., 2023] uses GPT-4 to generate architectures with natural-language guidance. LLMatic [Nasir et al., 2024] applies LLMs within a quality-diversity framework. FunSearch [Romera-Paredes et al., 2024] uses LLMs to discover novel mathematical functions, demonstrating the broader potential of LLM-guided search. GPT-NAS [Li et al., 2023] proposes using GPT models directly as architecture generators, and Evolution through Large Models [Lehman et al., 2024] provides a framework for LLM-guided evolutionary computation. Most of these approaches assume that iterative feedback improves generation quality, an assumption our work directly tests. A key gap in this literature is the absence of carefully controlled comparisons against random baselines and systematic ablations of feedback granularity.

Self-Refinement and Its Limits. The self-refinement paradigm was popularised by Madaan et al. [2023] and has been applied to code generation, reasoning, and creative tasks. Reflexion [Shinn et al., 2023] uses linguistic reinforcement signals to improve agent performance over multiple trials. However, a growing body of work challenges the universality of this paradigm. Huang et al. [2024] provided evidence that self-correction degrades performance when the model cannot reli-

ably evaluate its own outputs. Kamoi et al. [2024] surveyed conditions under which self-correction is unreliable, Olausson et al. [2024] showed limited effectiveness of self-repair in code generation, Tyen et al. [2024] demonstrated that LLMs struggle to identify reasoning errors even when they can sometimes correct them, and Stechly et al. [2024] documented self-verification failures in planning. Our results extend these findings to architecture design: even with ground-truth validation accuracy as feedback, a small LLM fails to leverage it effectively, suggesting the bottleneck lies in causal reasoning capacity rather than feedback quality.

AI-Assisted Scientific Discovery. The broader programme of using LLMs as research assistants [Lu et al., 2024, Si et al., 2024, Wang et al., 2024] motivates our study. Si et al. [2024] found that LLM-generated ideas are rated as more novel but less feasible than human ideas, suggesting strong priors but weak iterative refinement. Understanding when LLM proposals benefit from feedback is essential for designing effective human–AI collaboration.

3 Methodology

We design a controlled experiment with six conditions that isolate the effects of (i) using an LLM versus random generation, (ii) providing feedback versus zero-shot generation, and (iii) the structure of the feedback signal. All conditions share the same search space, training protocol, and evaluation procedure.

3.1 Search Space

Each architecture is a sequential convolutional neural network specified as a JSON object. The configurable components are: **number of blocks** (2–6); **convolution type** per block (standard 3×3 , depthwise-separable, dilated, or bottleneck); **channel widths** (powers of two from 16 to 256); **kernel sizes** (3×3 or 5×5); **activation functions** (ReLU, GELU, SiLU, or Mish); **normalisation** (batch, layer, group, or none); **pooling** (max, average, strided convolution, or none); **skip connections** (identity, projection, or none); and a **classifier head** (global average pooling followed by 1–2 linear layers with configurable dropout in $[0, 0.5]$). This space is intentionally broad enough to contain both good and poor designs while remaining small enough for exhaustive per-architecture training. Architectures are validated against a JSON schema before instantiation; the parser sanitises outputs to ensure structural validity. All conditions produce zero invalid architectures, eliminating validity as a confound. The complete schema is provided in Appendix B.

3.2 Experimental Conditions

We evaluate six conditions, each generating $n=20$ architectures per dataset:

Condition A: Random Search. Architectures are sampled uniformly at random from the search space. Each component is drawn independently from its marginal distribution.

Condition A2: Filtered Random Search. Identical to A, except architectures are validated against heuristic filters (minimum pooling layers, parameter bounds) before counting toward the budget. Designs failing the filter are discarded and resampled. This condition tests whether the LLM’s advantage stems solely from avoiding degenerate designs.

Condition B: LLM Zero-Shot. The LLM receives a system prompt describing the search space, the JSON schema, and a task description, then generates an architecture in a single forward pass with no performance feedback. Each of the 20 proposals uses a fresh context window.

Condition C: LLM + Unstructured Feedback. The LLM generates architectures sequentially within a single conversation. After each architecture is trained and evaluated on the validation set, the model receives a plain-text summary: “Your previous architecture achieved X% validation accuracy with Y parameters. Please design an improved architecture.”

Condition D: LLM + Structured Feedback. Identical to C, except the feedback includes a JSON report containing validation accuracy, training loss curve statistics (final loss, convergence rate), parameter count, and the full previous architecture specification.

Condition E: Regularised Evolution (REA). We implement the regularised evolutionary algorithm of [Real et al. \[2019\]](#), adapted to our search space. A population of size 10 is initialised randomly; at each step the best of 5 randomly sampled individuals is selected by *validation accuracy* as the parent, a single random mutation is applied, and the oldest individual is removed. We acknowledge that 20 evaluations is a modest budget for evolutionary search [[Stanley et al., 2019](#)]; REA is included as a standard reference point rather than a competitive evolutionary baseline. The bimodal variance (std=17.2 on CIFAR-10) reflects this small-population regime.

3.3 Training Protocol

All architectures are trained for 50 epochs on each dataset (CIFAR-10 and CIFAR-100 [[Krizhevsky, 2009](#)]) using SGD with momentum (0.9), cosine learning-rate annealing from 0.1 to 0.001, weight decay 5×10^{-4} , and batch size 128. Standard data augmentation is applied: random horizontal flips, random crops with 4-pixel padding, and Cutout [[DeVries and Taylor, 2017](#)] with a 16×16 mask. A fixed 90/10 train/validation split is used. Test-set evaluation is performed once per architecture after training; *validation accuracy is used for all feedback and selection decisions*. This separation ensures that test performance is never used to guide the search process.

3.4 LLM Configuration

We use Qwen3-1.7B [[Qwen Team, 2025](#)] loaded in 4-bit quantisation (NF4) on a single NVIDIA A100-40 GB. This model

Algorithm 1 Unified Architecture Evaluation

Require: Condition c , budget $N=20$, dataset $\mathcal{D}$

```

1: Split  $\mathcal{D} \rightarrow \mathcal{D}_{\text{trn}}$  (90%),  $\mathcal{D}_{\text{val}}$  (10%)
2:  $\mathcal{H} \leftarrow \emptyset$ 
3: for  $i = 1$  to  $N$  do
4:    $\alpha_i \leftarrow \text{PROPOSE}(c, \mathcal{H})$ 
5:   Train  $\alpha_i$  on  $\mathcal{D}_{\text{trn}}$  for 50 epochs
6:    $v_i \leftarrow \text{Evaluate}(\alpha_i, \mathcal{D}_{\text{val}})$ 
7:    $t_i \leftarrow \text{Evaluate}(\alpha_i, \mathcal{D}_{\text{test}})$ 
8:    $\mathcal{H} \leftarrow \mathcal{H} \cup \{(\alpha_i, v_i)\}$ 
9: end for
10:  $\alpha^* \leftarrow \arg \max_{(\alpha_i, v_i) \in \mathcal{H}} v_i$  {select by validation}
11: return  $t^*$  corresponding to  $\alpha^*$ , full history  $\mathcal{H}$ 
```

was chosen to reflect a practical cost-constrained scenario in which a researcher uses a small, locally deployable model for architecture generation. Generation uses temperature 0.7, top- p 0.9, and maximum output length 2048 tokens. The choice of a single small model is a deliberate limitation: the “narrow-band prior” finding may be model-size dependent, and larger models with richer priors may benefit more from feedback (Section 5.4). Prompt templates are provided in Appendix A.

3.5 Statistical Analysis Plan

We pre-specify seven key pairwise comparisons (Table 3) and apply Bonferroni correction [[Bonferroni, 1936](#)] with $\alpha_{\text{corr}}=0.05/7=0.0071$. Each comparison uses both Welch’s t -test (unequal variances) and the Mann–Whitney U test [[Mann and Whitney, 1947](#)] (non-parametric). We report a result as significant only when $p < \alpha_{\text{corr}}=0.0071$ and mark such results with *. Effect sizes are Cohen’s d [[Cohen, 1988](#)] with 95% bootstrap confidence intervals (10,000 resamples) [[Efron and Tibshirani, 1993](#)]. Diversity is measured by mean pairwise Jaccard distance [[Jaccard, 1912](#)] over design-choice vectors. Rank stability is assessed via Spearman’s ρ between validation accuracy at epoch 20 and final test accuracy at epoch 50.

3.6 Algorithm Overview

Algorithm 1 presents the unified evaluation framework. All conditions follow the same outer loop; only the PROPOSE function differs. The algorithm selects the best architecture by *validation* accuracy and evaluates it on the test set only once.

The PROPOSE function instantiates differently for each condition: A and A2 sample randomly (A2 applies heuristic filters and resamples); B queries the LLM with no history; C and D query the LLM with accumulated history $\mathcal{H}$; and E applies tournament selection and mutation.

3.7 Replication

All experiments use seed 42 as the primary seed. A partial replication with seed 137 ($n=20$) covers conditions A, A2, B, and C on CIFAR-10, and condition B on CIFAR-100, to verify the stability of the main findings.

Table 1: Test accuracy (%) on **CIFAR-10** ($n=20$, seed 42). Best mean in **bold**. Par. = parameters (K).

Condition	Mean $\pm$ Std	Med	Best	Par. (K)
A: Random	77.3 $\pm$ 22.6	84.9	90.3	508 $\pm$ 460
A2: Filt. Rand.	66.6 $\pm$ 32.8	84.9	90.3	778 $\pm$ 612
B: Zero-Shot	89.0 $\pm$ 0.7	89.0	89.8	511 $\pm$ 0
C: Unstr. FB	87.6 $\pm$ 1.9	88.7	90.2	574 $\pm$ 191
D: Struct. FB	87.2 $\pm$ 1.8	86.1	91.4	727 $\pm$ 302
E: Reg. Evol.	82.9 $\pm$ 17.2	86.6	90.6	625 $\pm$ 375

Table 2: Test accuracy (%) on **CIFAR-100** ($n=20$, seed 42).

Condition	Mean $\pm$ Std
A: Random	53.7 $\pm$ 6.1
A2: Filt. Rand.	49.0 $\pm$ 17.0
B: Zero-Shot	61.1 $\pm$ 2.7
C: Unstr. FB	61.6 $\pm$ 2.6
D: Struct. FB	60.9 $\pm$ 3.8
E: Reg. Evol.	58.6 $\pm$ 5.9

4 Results

4.1 Main Results

Tables 1 and 2 present the main accuracy results across all six conditions. Figure 1 shows the full distributions.

All three LLM conditions substantially outperform random search on both datasets, with dramatically lower variance. On CIFAR-10, LLM zero-shot achieves 89.0 ± 0.7 , compared to 77.3 ± 22.6 for random search, representing a 32-fold reduction in standard deviation. On CIFAR-100, the LLM conditions achieve means of 60.9–61.6, compared to 53.7 ± 6.1 for random search.

The filtered random baseline (A2) performs *worse* than unfiltered random (66.6 vs. 77.3 mean on CIFAR-10). This counterintuitive result arises because the heuristic filter inadvertently eliminates certain architecture families, including some with few pooling layers that happen to perform well [Tan and Le, 2019]. The filter shifts the parameter distribution toward larger models ($778 \pm 612\text{K}$ vs. $508 \pm 460\text{K}$; Table 4) without improving generalisation. This result confirms that the LLM’s advantage cannot be attributed solely to generating well-formed or appropriately sized designs, since a simple validity filter actually *degrades* random search performance.

Regularised evolution (E) exhibits high variance ($\text{std}=17.2$ on CIFAR-10) due to bimodal behaviour under the 20-evaluation budget: some runs inherit good designs from early mutations while others remain dominated by the random initial population. The best-case REA accuracy of 90.6% is competitive with all LLM conditions.

Crucially, feedback does not improve upon zero-shot generation. On CIFAR-10, both feedback conditions achieve lower mean accuracy than zero-shot (87.6 and 87.2 vs. 89.0), and the structured case is statistically significant (Section 4.2). On CIFAR-100, the three LLM conditions are statistically indistinguishable (B vs. C: $p=0.61$, $d=-0.16$; B vs. D: $p=0.81$, $d=0.08$), indicating that feedback neither helps nor hurts on the harder task.

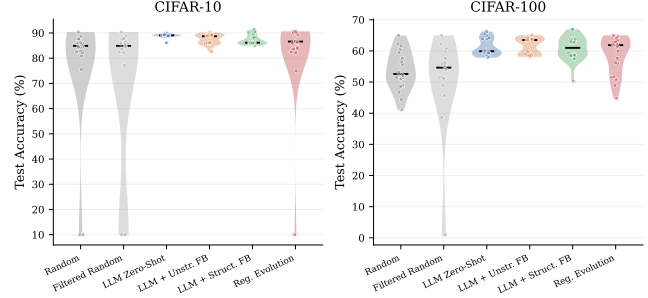


Figure 1: Violin plots with individual data points for all six conditions on CIFAR-10 (top) and CIFAR-100 (bottom). LLM conditions form tight distributions; random baselines and REA show wide, often bimodal spreads.

Table 3: Pre-specified pairwise comparisons ($k=7$, $\alpha_{\text{corr}}=0.0071$). Welch’s t , Mann–Whitney U , Cohen’s d [95% CI]. * significant after Bonferroni. Positive d : first condition higher.

CIFAR-10			
Comparison	p_t	p_U	d
A vs B	.037	<.001*	−0.71
A2 vs B	.008	<.001*	−0.94
B vs C	.007	.042	+0.93
B vs D	<.001*	.002*	+1.25
A vs E	.40	.066	−0.27
CIFAR-100			
A vs B	<.001*	<.001*	−1.53
B vs C	.606	.787	−0.16

4.2 Statistical Comparisons

Table 3 reports the seven pre-specified comparisons with Bonferroni-corrected threshold $\alpha_{\text{corr}}=0.0071$.

On CIFAR-10, the comparison between zero-shot (B) and structured feedback (D) yields a large effect size ($d=1.25$) that survives Bonferroni correction on both the t -test ($p<0.001$) and U -test ($p=0.002$). This is the strongest evidence that feedback hurts. The zero-shot vs. unstructured feedback comparison (B vs. C) shows a similar direction ($d=0.93$), but the p -values ($p_t=0.007$, $p_U=0.042$) place it at the margin: p_t approximately equals α_{corr} , and p_U does not survive correction. We therefore characterise B vs. C as *marginal* and B vs. D as *significant*. For random vs. zero-shot, the rank-based U -test ($p<0.001$) survives correction even when the t -test ($p=0.037$) does not, reflecting the heavy-tailed random-search distribution where rank-based tests have greater power. Random search versus REA shows no significant difference ($p_t=0.40$, $d=-0.27$).

On CIFAR-100, all LLM conditions significantly outperform random search with large effect sizes ($d=-1.53$ for A vs. B, $p<0.001$). Pairwise comparisons among LLM conditions yield no significant differences (B vs. C: $d=-0.16$, $p=0.61$), with confidence intervals crossing zero. The “feedback hurts” finding is therefore specific to CIFAR-10, where the LLM prior is already well-calibrated; on CIFAR-100 all LLM conditions perform comparably.

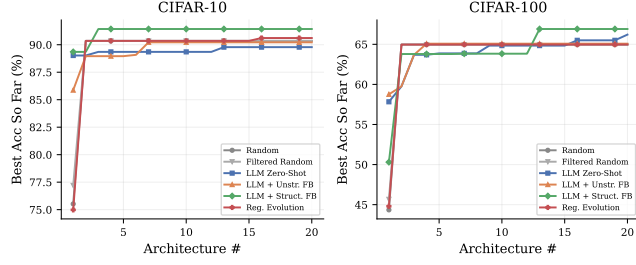


Figure 2: Best-so-far test accuracy across 20 evaluations. LLM conditions converge within 3–5 evaluations. Random and REA show staircase patterns.

Table 4: Architecture characteristics (CIFAR-10, seed 42). Par. = mean parameters (K). Blk = mean blocks. Gap = generalisation gap (%). Eff. = accuracy per M parameters. Div. = Jaccard distance.

Cond.	Par.	Blk	Gap	Eff.	Div.	Time (s)
A	508	4.2	4.2	355	.856	409
A2	778	4.3	3.9	213	.853	397
B	511	4.0	6.0	174	.022	219
C	574	4.0	6.4	162	.187	223
D	727	4.0	6.8	137	.268	236
E	625	4.7	6.5	215	.730	351

4.3 Convergence and Diversity

Figure 2 shows best-so-far accuracy as a function of evaluation number. LLM zero-shot reaches near-optimal performance within 3–5 evaluations and shows minimal improvement thereafter, consistent with its low diversity. Random search and REA exhibit staircase improvement patterns. The feedback conditions converge at an intermediate speed but never surpass the zero-shot mean on CIFAR-10.

Table 4 reveals the most striking finding. The zero-shot condition exhibits a Jaccard distance of only 0.022, indicating near-identical architectures. The parameter standard deviation is 0K: every architecture uses the same number of blocks ($K=4$), the same convolution types, the same activation, the same normalisation, and the same skip-connection configuration. This is 39 times lower diversity than random search (0.856). Yet this near-deterministic condition achieves the highest mean accuracy on CIFAR-10.

Figure 3 details the specific design choices. The zero-shot condition (B) exclusively selects standard 3×3 convolutions (100%), ReLU activations (100%), and BatchNorm normalisation (100%), with skip connections split evenly between projection (50%) and identity (50%). This template closely resembles the classical ResNet block [He et al., 2016, Nair and Hinton, 2010, Ioffe and Szegedy, 2015] rather than more modern depthwise-separable or GELU-based designs [Howard et al., 2017, Ramachandran et al., 2017]. The LLM’s prior is not only narrow but *conservative*.

Unstructured feedback (C) maintains 100% standard 3×3 and 100% ReLU but shifts skip connections toward projection (75% vs. 25% identity). Structured feedback (D) introduces marginal variation: 99% standard 3×3 with 1% other types, 94% ReLU with 6% GELU, and 100% BatchNorm. By con-

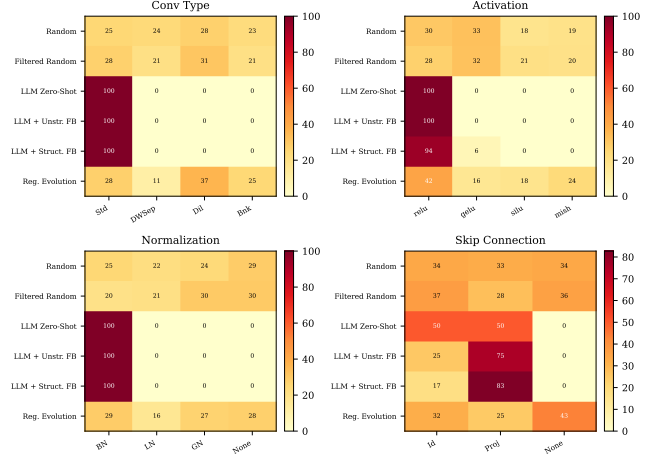


Figure 3: Design choice frequencies by condition. Zero-shot (B) concentrates entirely on standard 3×3 convolutions, ReLU, and BatchNorm. Feedback conditions show slight broadening; random and REA distribute uniformly.

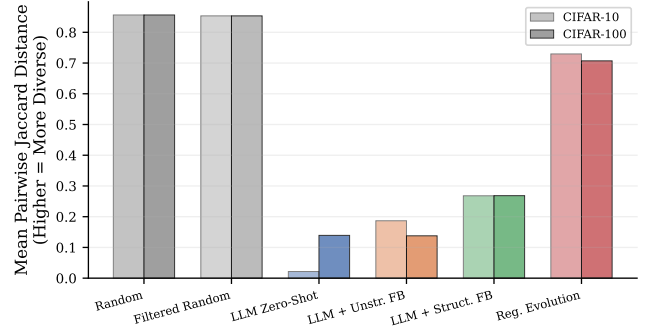


Figure 4: Mean pairwise Jaccard distance by condition. Feedback progressively increases diversity without improving accuracy.

trast, random search distributes roughly uniformly across convolution types (28% dilated, 25% standard, 24% depthwise-separable, 23% bottleneck) and activations (33% GELU, 30% ReLU, 19% Mish, 18% SiLU). REA produces a mixed distribution (37% dilated, 28% standard, 11% depthwise-separable, 25% bottleneck; 42% ReLU, 24% Mish, 16% GELU, 18% SiLU).

Feedback increases diversity monotonically (Figure 4): unstructured feedback raises the Jaccard distance to 0.187 (8.5-fold over zero-shot), and structured feedback further increases it to 0.268. This diversification occurs because feedback prompts the LLM to deviate from its default template. However, these deviations move designs away from the strong prior and into less well-characterised regions, resulting in lower mean accuracy on CIFAR-10. We term this the *diversity–accuracy inversion*: the opposite of what is typically assumed in search-based NAS, where greater diversity aids exploration.

Figure 5 illustrates the parameter–accuracy relationship. Feedback conditions produce progressively larger architectures (B: 511K, C: 574K, D: 727K), yet accuracy decreases. The efficiency metric (accuracy per million parameters) in Table 4 confirms this: condition D achieves the lowest efficiency

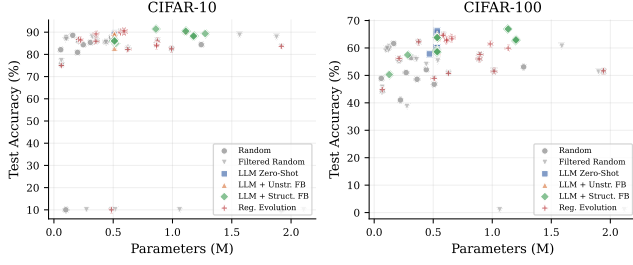


Figure 5: Parameters vs. test accuracy. LLM conditions cluster tightly at 400–700K; random conditions span a much wider range.

Table 5: Rank stability: Spearman ρ between validation accuracy at epoch 20 and final test accuracy (epoch 50).

Condition	Spearman ρ	p
A: Random	0.865	<0.001
A2: Filt. Rand.	0.890	<0.001
B: Zero-Shot	0.462	0.040
C: Unstr. FB	0.708	<0.001
D: Struct. FB	0.651	0.002
E: Reg. Evol.	0.878	<0.001

(137) despite having the most parameters.

4.4 Rank Stability and Training Dynamics

Table 5 reports Spearman rank correlations between validation accuracy at epoch 20 and final test accuracy. Random search and REA exhibit high rank stability ($\rho > 0.86$), indicating that early-stopping proxies reliably identify the best architectures. The zero-shot condition shows lower stability ($\rho = 0.462$, $p = 0.040$), but this does not indicate poor architecture quality; rather, it reflects the narrow accuracy range of zero-shot designs (all between 87.8% and 89.8%), where small training fluctuations reorder rankings. The moderate ρ for B suggests caution regarding proposals to use early-epoch accuracy as a cheaper feedback signal for LLM-designed architectures, since the signal-to-noise ratio is low when the population is homogeneous.

Figure 6 shows training dynamics for the best architecture from each condition. All conditions achieve similar final training loss, with LLM architectures converging slightly faster. However, the generalisation gap (Table 4) is larger for LLM conditions (6.0–6.8%) than for random search (4.2%), suggesting the LLM’s template may benefit from stronger regularisation.

4.5 Transcript Analysis

A key hypothesis was that the LLM would produce explicit causal reasoning about architecture–performance relationships. Analysis of all transcripts reveals *zero* causal attributions: Qwen3-1.7B outputs JSON specifications directly with no accompanying reasoning text. This null result is consistent with the model’s small size and limited chain-of-thought capacity [Wei et al., 2022, Valmeekam et al., 2023]. Without causal attribution, the model cannot translate performance

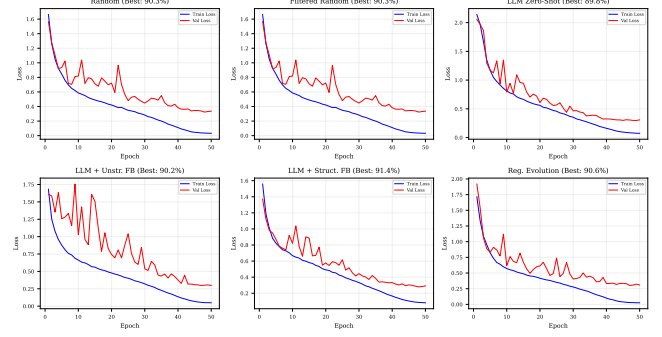


Figure 6: Training curves for the best architecture per condition (CIFAR-10). LLM architectures converge faster but exhibit a larger generalisation gap (Table 4).

metrics into targeted modifications, which may explain the feedback interference effect.

4.6 Replication

Partial replication with seed 137 ($n=20$) on CIFAR-10 confirms the main findings. LLM zero-shot achieves 88.2 ± 1.5 (vs. 89.0 ± 0.7 at seed 42), unstructured feedback achieves 88.3 ± 1.3 (vs. 87.6 ± 1.9), and random search achieves 69.3 ± 30.1 (vs. 77.3 ± 22.6). On CIFAR-100, the seed-137 replication of zero-shot yields 61.1 ± 2.5 (vs. 61.1 ± 2.7 at seed 42). Pooling seeds 42 and 137 ($n=40$) gives zero-shot 88.6 ± 1.2 and unstructured feedback 87.9 ± 1.7 on CIFAR-10, confirming a stable advantage. The LLM’s superiority over random search and the absence of feedback benefit replicate across seeds.

5 Discussion

5.1 The LLM as a Narrowband Prior

Our results reframe the role of the LLM in architecture search. Rather than functioning as a flexible search agent that explores the design space and learns from feedback, the LLM behaves as a *strong but narrow prior*: it concentrates probability mass on a single well-performing template and resists meaningful exploration. The zero structural variance of condition B (parameter std = 0K, Jaccard distance 0.022) is remarkable. Twenty independent queries produce identical design choices: standard 3×3 convolutions, ReLU, BatchNorm, and mixed projection/identity skips. This template closely resembles a canonical ResNet block [He et al., 2016], suggesting the LLM’s prior is shaped by the dominant architecture family in its training data.

The prior is well-calibrated in the sense that the default template achieves high accuracy on both CIFAR-10 and CIFAR-100, but it is brittle: perturbations induced by feedback degrade rather than improve performance on the easier task. The practical implication is clear: rather than investing compute in sequential refinement, practitioners should sample a small number (3–5) of independent zero-shot architectures and se-

lect the best by validation accuracy.

5.2 Why Does Feedback Hurt?

We identify three mechanisms for the feedback interference effect. First, the LLM may lack the causal reasoning capacity to translate performance metrics into actionable modifications. The null transcript analysis supports this: without explicit reasoning, the model processes feedback as a generic signal to “change something” rather than targeted guidance.

Second, feedback may trigger a *novelty bias* in which the instruction to “improve” is interpreted as “change.” The diversity data support this: Jaccard distance increases monotonically from zero-shot (0.022) through unstructured (0.187) to structured feedback (0.268). More information in the feedback paradoxically leads to more deviation from the prior.

Third, with multiple components changing between iterations and only 20 evaluations, the LLM cannot isolate which changes caused performance differences. Structured edit prompts constraining the model to change at most one component per iteration [Madaan et al., 2023] could mitigate this confound and represent a promising direction for future work.

5.3 Training Budget Confound

An important caveat concerns the fixed training budget. Feedback conditions produce larger architectures (B: 511K, C: 574K, D: 727K), which may require more epochs to converge. If anything, this bias favours feedback conditions: larger models have greater capacity and should benefit from 50 epochs of training. Yet accuracy still decreases. The generalisation gap data (Table 4) show that larger LLM architectures overfit more (6.8% gap for D vs. 6.0% for B), indicating the accuracy loss is not explained by underfitting alone. Nevertheless, the confound is real: future work should explore adaptive training schedules that account for model size.

5.4 Limitations

Several limitations qualify our findings. **(i) Single small LLM.** We use Qwen3-1.7B-4bit; larger models with stronger reasoning capabilities [OpenAI, 2023] may respond differently to feedback and produce richer priors. The narrowband prior finding may be model-size dependent, with larger models exhibiting broader priors that benefit more from refinement. **(ii) Search space scope.** Our custom CNN space on CIFAR-10/100 provides maximal experimental control but limits generalisability. Evaluation on NAS-Bench-201 [Dong and Yang, 2020] would enable zero-noise comparisons and is a natural next step. **(iii) REA budget.** Twenty evaluations is modest for evolutionary search; the high REA variance reflects this limitation. **(iv) Transcript null result.** The small model’s minimal text output prevents us from characterising its reasoning process. **(v) Prompt design.** We do not explore structured edit prompts (e.g., “change at most one component”) that might preserve the LLM’s prior while enabling targeted exploration.

(vi) Fixed 50-epoch budget. Larger feedback-condition architectures may benefit from longer training. **(vii) Partial replication.** Seed 137 covers four of six conditions; full multi-seed replication would strengthen the conclusions.

6 Conclusion

We have presented a controlled study of LLM-guided neural architecture design that isolates the effect of performance feedback on generation quality. A small, quantised LLM (Qwen3-1.7B-4bit) encodes a strong prior over a single conservative template (standard 3×3 convolutions, ReLU, BatchNorm), producing 20 structurally identical designs that significantly outperform random search (89.0 vs. 77.3 on CIFAR-10, 61.1 vs. 53.7 on CIFAR-100) while reducing variance by more than 30-fold. Structured feedback significantly disrupts this prior on CIFAR-10 ($d=1.25$, $p<0.001$ after Bonferroni correction), increasing diversity without improving accuracy. On CIFAR-100, feedback and zero-shot perform comparably ($d=-0.16$, $p=0.61$). These findings challenge the assumption that self-refinement universally improves LLM-based design and suggest that, under fixed compute budgets, parallel zero-shot sampling is the preferred strategy. Future work should investigate whether larger models can productively leverage feedback, whether structured edit constraints preserve the prior while enabling exploration, and whether these findings generalise to cell-based and transformer search spaces.

Impact Statement

This work studies the automation of neural architecture design using language models. Our finding that zero-shot generation often suffices suggests a path toward more compute-efficient design workflows, reducing the environmental cost of NAS. We see no direct negative societal impacts from this methodology research, but acknowledge that improved NAS techniques could accelerate development of AI systems requiring careful ethical consideration.

Reproducibility

All code, prompts, configuration files, random seeds, and analysis scripts will be released in a public repository upon acceptance. The complete JSON schema, prompt templates, and per-architecture results are provided in the appendices.

References

- Carlo E. Bonferroni. Teoria statistica delle classi e calcolo delle probabilit . *Pubblicazioni del R Istituto Superiore di Scienze Economiche e Commerciali di Firenze*, 8:3–62, 1936.
- Angelica Chen, David Dohan, and David So. EvoPrompting: Language models for code-level neural architecture search.

- In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.
- Jacob Cohen. *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum Associates, 2nd edition, 1988. ISBN 978-0805802832.
- Terrell DeVries and Graham W. Taylor. Improved regularization of convolutional neural networks with cutout. *arXiv preprint arXiv:1708.04552*, 2017. URL <https://arxiv.org/abs/1708.04552>.
- Xuanyi Dong and Yi Yang. NAS-Bench-201: Extending the scope of reproducible neural architecture search. In *Proceedings of the 8th International Conference on Learning Representations (ICLR)*, 2020. URL <https://arxiv.org/abs/2001.00326>.
- Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1993. doi: 10.1007/978-1-4899-4541-9.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016. doi: 10.1109/CVPR.2016.90.
- Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. MobileNets: Efficient convolutional neural networks for mobile vision applications. 2017. URL <https://arxiv.org/abs/1704.04861>.
- Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. 2024. URL <https://arxiv.org/abs/2310.01798>.
- Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 448–456. PMLR, 2015.
- Paul Jaccard. The distribution of the flora in the alpine zone. *New Phytologist*, 11(2):37–50, 1912.
- Ryo Kamoi, Yusen Raman, Guanghui Peng, Hendrik Strobelt, and Bradley Malin. When can LLMs actually correct their own mistakes? A critical survey of self-correction of LLMs. *Transactions on Machine Learning Research*, 2024.
- Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.
- Joel Lehman, Jonathan Gordon, Sherjil Jain, Kamal Ndousse, Cathy Yeh, and Kenneth O. Stanley. Evolution through large models. *arXiv preprint arXiv:2206.08896*, 2024. URL <https://arxiv.org/abs/2206.08896>.
- Haixin Li, Rongyu Jin, Jianan He, Wentao Zheng, Zhuoran Wei, and Siyuan Chi. GPT-NAS: Neural architecture search with the generative pre-trained model. *arXiv preprint arXiv:2305.05351*, 2023. URL <https://arxiv.org/abs/2305.05351>.
- Liam Li and Ameet Talwalkar. Random search and reproducibility for neural architecture search. In *Proceedings of the 36th Conference on Uncertainty in Artificial Intelligence (UAI)*, pages 367–377. PMLR, 2020.
- Hanxiao Liu, Karen Simonyan, and Yiming Yang. DARTS: Differentiable architecture search. In *Proceedings of the 7th International Conference on Learning Representations (ICLR)*, 2019. URL <https://arxiv.org/abs/1806.09055>.
- Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024. URL <https://arxiv.org/abs/2408.06292>.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36, 2023.
- Henry B. Mann and Donald R. Whitney. On a test of whether one of two random variables is stochastically larger than the other. *Annals of Mathematical Statistics*, 18(1):50–60, 1947.
- Vinod Nair and Geoffrey E. Hinton. Rectified linear units improve restricted Boltzmann machines. In *Proceedings of the 27th International Conference on Machine Learning (ICML)*, pages 807–814, 2010.
- Muhammad U. Nasir, Sam Earle, Julian Togelius, Steven James, and Christopher Cleghorn. LLMatic: Neural architecture search via large language models and quality-diversity optimization. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO)*, pages 1110–1118, 2024.
- Theo X. Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. Is self-repair a silver bullet for code generation? *arXiv preprint arXiv:2306.09896*, 2024. URL <https://arxiv.org/abs/2306.09896>.
- OpenAI. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023. URL <https://arxiv.org/abs/2303.08774>.

- Hieu Pham, Melody Y. Guan, Barret Zoph, Quoc V. Le, and Jeff Dean. Efficient neural architecture search via parameter sharing. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 4095–4104. PMLR, 2018.
- Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. URL <https://arxiv.org/abs/2505.09388>.
- Prajit Ramachandran, Barret Zoph, and Quoc V. Le. Searching for activation functions. *arXiv preprint arXiv:1710.05941*, 2017. URL <https://arxiv.org/abs/1710.05941>.
- Esteban Real, Alok Aggarwal, Yanping Huang, and Quoc V. Le. Regularized evolution for image classifier architecture search. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 4780–4789, 2019. doi: 10.1609/aaai.v33i01.33014780.
- Bernardino Romera-Paredes, Mohammadamin Barekatin, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024. doi: 10.1038/s41586-023-06924-6.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.
- Chenglei Si, Diyi Yang, and Tatsunori Hashimoto. Can LLMs generate novel research ideas? A large-scale human study with 100+ NLP researchers. *arXiv preprint arXiv:2409.04109*, 2024. URL <https://arxiv.org/abs/2409.04109>.
- David R. So, Quoc V. Le, and Chen Liang. The evolved transformer. pages 5877–5886, 2019.
- Kenneth O. Stanley, Jeff Clune, Joel Lehman, and Risto Miikkulainen. Designing neural networks through neuroevolution. *Nature Machine Intelligence*, 1(1):24–35, 2019. doi: 10.1038/s42256-018-0006-z.
- Kaya Stechly, Karthik Valmeekam, and Subbarao Kambhampati. On the self-verification limitations of large language models on reasoning and planning tasks. *arXiv preprint arXiv:2402.08115*, 2024. URL <https://arxiv.org/abs/2402.08115>.
- Mingxing Tan and Quoc V. Le. EfficientNet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 6105–6114. PMLR, 2019.
- Gladys Tyen, Hassan Mansoor, Peter Chen, Tony Muresanu, and Jason Phang. LLMs cannot find reasoning errors, but can correct them! *Findings of the Association for Computational Linguistics: ACL*, 2024.
- Karthik Valmeekam, Matthew Marquez, Sarath Sreedharan, and Subbarao Kambhampati. On the planning abilities of large language models – a critical investigation. *Advances in Neural Information Processing Systems*, 36, 2023.
- Ziru Wang, Fangqi Chen, Yubo Arber, et al. ScienceAgent-Bench: Toward rigorous assessment of language agents for data-driven scientific discovery. *arXiv preprint arXiv:2410.05080*, 2024. URL <https://arxiv.org/abs/2410.05080>.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 24824–24837, 2022.
- Colin White, Mahmoud Safari, Rhea Suber, Binxin Ru, Shan Thomas, and Frank Hutter. Neural architecture search: Insights from 1000 papers. *arXiv preprint arXiv:2301.08727*, 2023. URL <https://arxiv.org/abs/2301.08727>.
- Antoine Yang, Pedro M. Esperanca, and Fabio M. Carlucci. NAS evaluation is frustratingly hard. In *Proceedings of the 8th International Conference on Learning Representations (ICLR)*, 2020. URL <https://arxiv.org/abs/1912.12522>.
- Chris Ying, Aaron Klein, Eric Christiansen, Esteban Real, Kevin Murphy, and Frank Hutter. NAS-Bench-101: Towards reproducible neural architecture search. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 7105–7114. PMLR, 2019.
- Mingkai Zheng, Xiu Su, Shan You, Fei Wang, Chen Qian, Chang Xu, and Samuel Albanie. Can GPT-4 perform neural architecture search? *arXiv preprint arXiv:2304.10970*, 2023. URL <https://arxiv.org/abs/2304.10970>.
- Barret Zoph and Quoc V. Le. Neural architecture search with reinforcement learning. In *Proceedings of the 5th International Conference on Learning Representations (ICLR)*, 2017. URL <https://arxiv.org/abs/1611.01578>.

A Prompt Templates

A.1 System Prompt (All LLM Conditions)

You are an expert neural network architect. Your task is to design CNN architectures for image classification. You must output a valid JSON object conforming to the schema below. Design architectures that balance accuracy, parameter efficiency, and trainability.

The system prompt is followed by the full JSON schema (Appendix B) and a task description specifying the target dataset and input dimensions.

A.2 Unstructured Feedback (Condition C)

After each evaluation:

Your previous architecture achieved {val_acc} validation accuracy with {params}K parameters. Please design an improved architecture.

A.3 Structured Feedback (Condition D)

A JSON report is appended containing: val_accuracy, train_loss_final, convergence_rate, param_count and the full architecture specification of the previous design.

B JSON Schema

The architecture schema includes: num_blocks (2–6), blocks (array, each with conv_type, channels, kernel_size, activation, norm, pooling), skip_type (identity/projection/none), classifier (num_layers, dropout), and global_pooling (avg/max). Convolution types: standard, depthwise-separable, dilated, bottleneck. Activations: ReLU, GELU, SiLU, Mish. Normalisation: batch, layer, group, none.

C Full Statistical Tables

The seven pre-specified comparisons and their full statistics are reported in Table 3. Additional exploratory comparisons include: A vs C on CIFAR-100 ($p_t < 0.001$, $p_U < 0.001$, $d = -1.63$, significant), B vs D on CIFAR-100 ($p_t = 0.81$, $p_U = 0.39$, $d = 0.08$, not significant), and A vs E on CIFAR-10 ($p_t = 0.40$, $p_U = 0.066$, $d = -0.27$, not significant).

D Per-Seed Breakdown

Seed-137 results confirm all qualitative findings. Random conditions show high variance ($\text{std} > 27$) across seeds, and the LLM advantage persists. Pooled zero-shot CIFAR-10 (88.6 ± 1.2 , $n = 40$) confirms the stability of the main finding.

Table 6: Per-seed results for replicated conditions.

Condition	Dataset	Seed	Mean $\pm$ Std
A: Random	C-10	42	77.3 $\pm$ 22.6
A: Random	C-10	137	69.3 $\pm$ 30.1
B: Zero-Shot	C-10	42	89.0 $\pm$ 0.7
B: Zero-Shot	C-10	137	88.2 $\pm$ 1.5
C: Unstr. FB	C-10	42	87.6 $\pm$ 1.9
C: Unstr. FB	C-10	137	88.3 $\pm$ 1.3
B: Zero-Shot	C-100	42	61.1 $\pm$ 2.7
B: Zero-Shot	C-100	137	61.1 $\pm$ 2.5
<i>Pooled ($n = 40$, seeds 42+137)</i>			
B: Zero-Shot	C-10	all	88.6 $\pm$ 1.2
C: Unstr. FB	C-10	all	87.9 $\pm$ 1.7
B: Zero-Shot	C-100	all	61.1 $\pm$ 2.6
A: Random	C-10	all	73.3 $\pm$ 26.9

E Channel Width Distributions

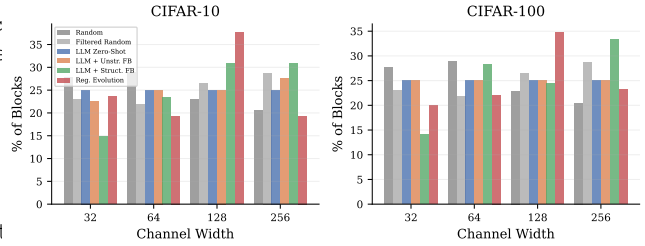


Figure 7: Channel width distributions across conditions. LLM conditions concentrate on 32–128 channels; random conditions span the full 16–256 range.